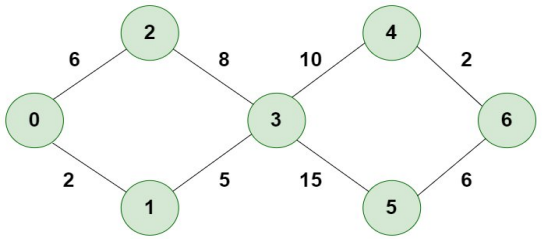
Dijkstra's Algorithm

What is Dijkstra’ s Algorithm

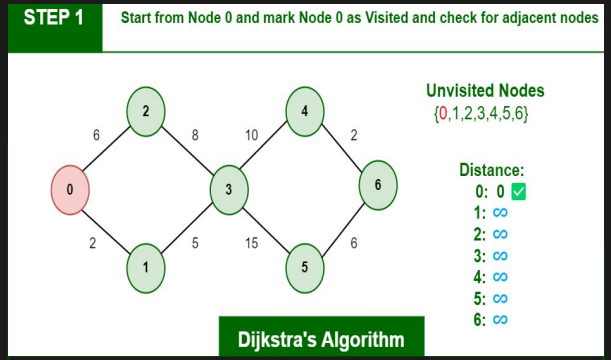
Dijkstra’s Algorithm was introduced by a dutch computer scientist Edsger W. Dijkstra in 1956. It is one of the popular algorithms in graph theory used to find the shortest path from a single source to all other vertices in a graph with non-negative weights.

How does it work?

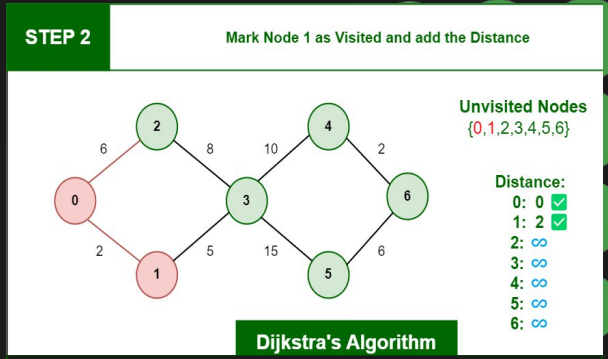
The algorithm works by maintaining a set of visited and unvisited vertices.
Starting from the source it repeatedly picks the vertex with the smallest tentative distance, updates the distances of its neighbours and continues until all vertices are processed or the destination is reached.



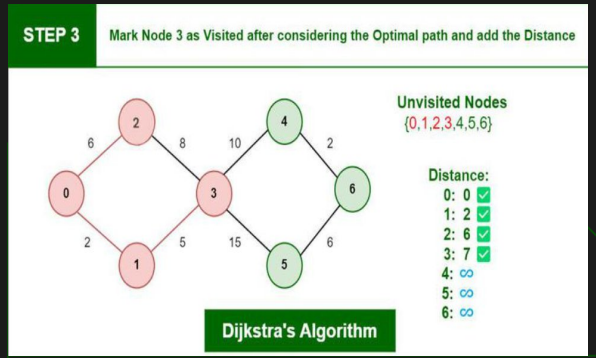
- The algorithm will generate the shortest path from node 0 to all the other nodes in the graph.
- The distance from the source node to all other node is unknown so we mark all of them as infinity.



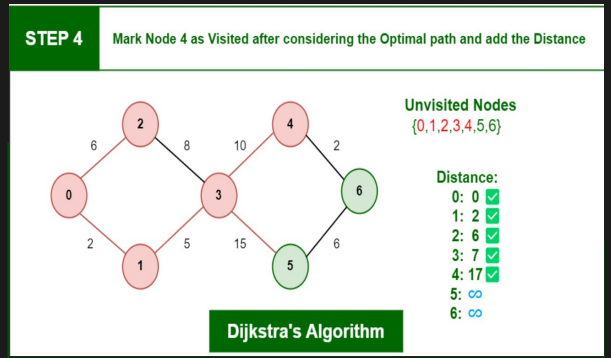
- Start from Node 0 and mark Node as visited as you can check in the image , visited Node is marked red.



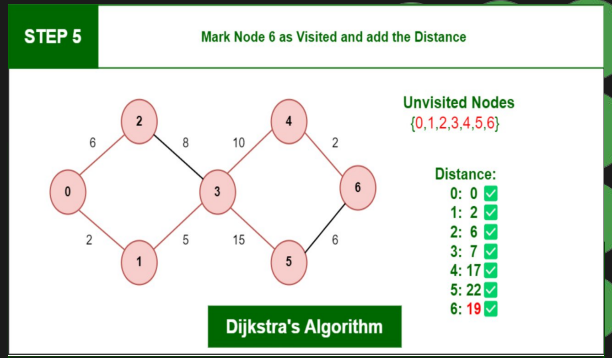
- Now we have to choices and choose Node with minimum distance.
- In this step **Node 1** is with Minimum distance adjacent Node, so marked it as visited and add up the distance.
- **Distance: Node 0 -> Node 1 = 2**



- Then Move Forward and check for adjacent Node which is Node 3, so marked it as visited and add up the distance, Now the distance will be:
- **Distance: Node 0 -> Node 1 -> Node 3 = 2 + 5 = 7**

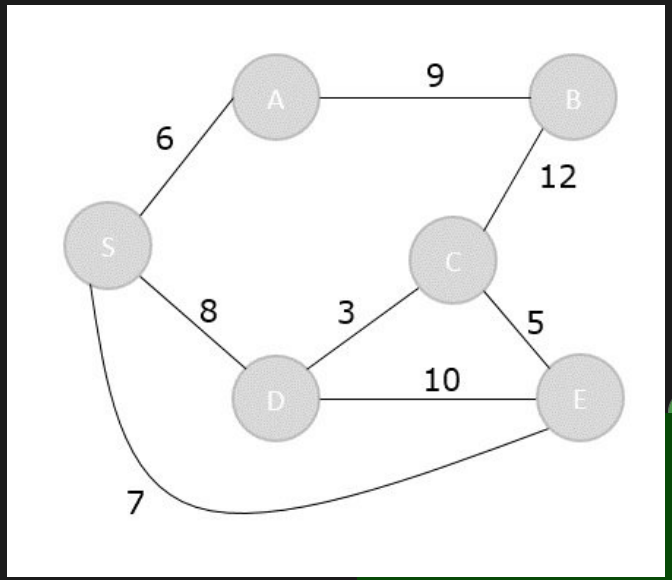


- Again we have two choices for adjacent Nodes, so choose Node with minimum distance.
- In this step **Node 4** is Minimum distance adjacent Node, so marked it as visited and add up the distance.
- **Distance: Node 0 -> Node 1 -> Node 3 -> Node 4 = 2 + 5 + 10 = 17**



- Again, Move Forward and check for adjacent Node which is **Node 6**, so marked it as visited and add up the distance, Now the distance will be:
- **Distance: Node 0 -> Node 1 -> Node 3 -> Node 4 -> Node 6 = 2 + 5 + 10 + 2 = 19**

Technical Explanation



Algorithm

Declare two arrays – distance[] to store the distances from the source vertex to the other vertices in graph and visited[] to store the visited vertices.

Set distance[S] to 0 and distance[v] = ∞, where v represents all the other vertices in the graph.

Add S to the visited[] array and find the adjacent vertices of S with the minimum distance.

The adjacent vertex to S, say A, has the minimum distance and is not in the visited array yet. A is picked and added to the visited array and the distance of A is changed from ∞ to the assigned distance of A, say d_1, where $d_1 < \infty$.

Repeat the process for the adjacent vertices of the visited vertices until the shortest path spanning tree is formed.

Pseudo Code

```
Function Dijkstra(Graph G, int StartNode, int NumNodes):
    // 1. Initialization
    int distances[MAX_NODES]
    int visited[MAX_NODES]

    // Initialize distances and visited arrays
    for i from 0 to NumNodes - 1:
        distances[i] = INFINITY
        visited[i] = FALSE

    distances[StartNode] = 0


// PQ stores (distance, node_index). We'll conceptually 'push'
here.
    PQ = InitializePriorityQueue()
    PQ_push(PQ, (0, StartNode))
```

```
  // 2. Main Loop
  while PQ is not empty:
      // Extract the node 'u' with the minimum distance 'd'
      (d, u) = PQ_pop_min(PQ)

      if visited[u] == TRUE:
          continue

      visited[u] = TRUE

// 3. Relaxation Step
      // Iterate over neighbors 'v' of 'u' in the graph G
      for each Edge (u, v, weight w) in G:
          newDistance = d + w

          if newDistance < distances[v]:
              distances[v] = newDistance
              // Update or insert the new shorter path into the PQ
              PQ_push(PQ, (newDistance, v))

  // 4. Result
  return distances
```

Applications Of Dijkstra's Algorithm

Digital Mapping Services

As there are various routes/paths connecting them but it has to show the minimum distance, so Dijkstra's Algorithm is

Social Networking Applications

If the social networking graph is very small, it uses standard Dijkstra's algorithm along with some other features to find the shortest paths, however, when the graph is becoming bigger, the standard algorithm takes a few seconds to count so,advanced algorithms are used.

Telephone Network

If we imagine a city to be a graph, the vertices represent the switching stations, and the edges represent the transmission lines and the

IP routing to find OSPF

Dijkstra's algorithm is widely used in the routing protocols required by the routers to update their forwarding table. The algorithm

Robotic Path

When the source and destination is known, the robot/drone moves in the ordered direction by following the shortest path to keep delivering

Advantages

Guarantees Shortest Path

It is guaranteed to find the shortest path from the source node to all other reachable nodes in a graph, provided all edge weights are non-negative.

Wide Applicability

It is used in numerous real-world applications, such as network routing protocols (like OSPF), GPS navigation systems, and telecommunications.

Relatively Efficient

When implemented with a priority queue (like a min-heap), its time complexity can be as efficient as O((V + E)log V), where V is the number of vertices and E is the number of edges.

Forms a shortest-path tree

When the algorithm finishes, the set of shortest paths from the source to all reachable vertices forms a shortest-path tree. This provides a structured representation of the shortest routes, which is useful in many applications, such as network topology analysis.

Disadvantages

Cannot Handle Negative Weights

The algorithm fails to produce correct results if the graph contains negative edge weights. This is because it assumes that once a node's shortest path is finalized, it will not be revisited or improved by a path involving a negative edge.

Blind Search Nature

It explores the graph without specific knowledge of the target node's location, which can sometimes lead to exploring unnecessary paths if only a single destination is desired and a more informed search algorithm (like A*) could be used.

Finds Shortest Paths from One Source Only

It is designed to find shortest paths from a single source node. For finding shortest paths between all pairs of nodes, algorithms like Floyd-Warshall might be more suitable.

Can Be Slow for Very Large Graphs

While efficient for many cases, its performance can degrade for extremely large graphs, especially if not implemented with an optimized priority queue.

Conclusion

Dijkstra’s Algorithm efficiently finds shortest paths

Fundamental in graph theory and networking

Basis for advanced algorithms like A* and Floyd–Warshall

Thank you….